

BAB II

Application Programming Interface (API)

2.1 Tujuan

1. Praktikan dapat mengimplementasikan arsitektur RESTful API dengan benar untuk mengelola data.
2. Praktikan dapat membuat server *backend* menggunakan Node.js dan Express.js untuk mengelola permintaan dan respons.
3. Praktikan dapat menghubungkan dan berinteraksi dengan *database* Supabase sebagai *backend-as-a-service*.
4. Praktikan dapat memisahkan logika aplikasi ke dalam model, *controller*, dan *router* untuk struktur kode yang terorganisir.
5. Praktikan dapat menghasilkan API yang terstruktur rapi, menggunakan variabel lingkungan, dan siap untuk di-*deploy*.

2.2 Alat dan Bahan

2.2.1 Laptop

Laptop adalah komputer pribadi portabel yang mengintegrasikan komponen utama seperti CPU, GPU, RAM, penyimpanan jangka panjang, serta sistem operasi dalam satu perangkat yang hemat energi. Layar LCD beresolusi tinggi dengan kepadatan piksel semakin umum digunakan, memungkinkan tampilan grafis yang tajam. Laptop biasanya dilengkapi dengan perangkat *input* bawaan seperti *keyboard*, *touchpad*, kamera, dan mikrofon, serta *port* I/O (misalnya USB, HDMI, atau Lightning) untuk koneksi ke perangkat eksternal. Selain itu, daya laptop ditopang oleh baterai isi ulang dan adaptor AC, dengan ketahanan baterai yang bervariasi sesuai desain. Beberapa model modern, seperti *hybrid* dan konvertibel, memungkinkan layar dilepas dan berfungsi sebagai tablet, memberikan fleksibilitas penggunaan.



Gambar 2. 1 Laptop

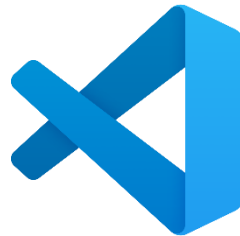
Secara teknis, laptop memiliki berbagai variasi spesifikasi untuk menyesuaikan kebutuhan pengguna. Prosesornya bisa terdiri dari dua hingga delapan inti dengan performa yang beragam, sementara kapasitas RAM biasanya 4–16 GB, sering kali disolder langsung pada *motherboard* sehingga tidak dapat di-*upgrade*. Faktor lain yang membedakan jenis laptop adalah ukuran fisik, resolusi layar, desain bodi (misalnya *subnotebook*, *netbook*, atau *ultrabook*), serta aksesoris tambahan seperti *keyboard* lepas-pasang. Laptop gaming umumnya membutuhkan GPU dengan performa tinggi serta layar beresolusi besar, sedangkan laptop bisnis menekankan portabilitas dan memori yang cukup besar. Dibandingkan dengan desktop, laptop lebih ringkas, hemat energi, dan memiliki daya portabilitas, meskipun biasanya mengorbankan fleksibilitas *upgrade* perangkat keras.

(Sumber: <https://www.techtarget.com/searchmobilecomputing/definition/laptop-computer>)

2.2.2 Visual Studio Code

Visual Studio Code adalah editor kode sumber terbuka yang ringan namun *powerful*, dikembangkan oleh Microsoft dan tersedia lintas platform (Windows, macOS, Linux, dan Web). VS Code mendukung hampir semua bahasa pemrograman populer, seperti JavaScript,

TypeScript, Python, C#, C++, Java, PHP, dan banyak lainnya melalui *Extension Marketplace*. Fitur bawaan yang penting meliputi *integrated terminal*, *debugger*, *Git version control*, *local history*, serta dukungan tema dan personalisasi antarmuka. Pengguna dapat menyinkronkan pengaturan antar perangkat menggunakan *Settings Sync* dan mengatur profil yang berbeda sesuai kebutuhan proyek.



Gambar 2. 2 Visual Studio Code

Selain fitur dasar, VS Code kini dilengkapi dengan *AI-assisted development* seperti GitHub Copilot dan Agent Mode, yang mampu memahami konteks kodebase, memberikan saran perbaikan, bahkan menjalankan perintah terminal secara otomatis untuk menyelesaikan tugas multi-langkah. Dukungan *Model Context Protocol* (MCP) dan integrasi ekstensi seperti Python, Jupyter, GitLens, Stripe, MongoDB, hingga *Remote Development* menjadikan VS Code fleksibel untuk berbagai *workflow*. Dengan opsi untuk digunakan secara lokal atau melalui *cloud* (misalnya GitHub Codespaces dan vscode.dev), VS Code telah menjadi salah satu lingkungan pengembangan paling serbaguna, mendukung produktivitas individu maupun kolaborasi tim lintas platform.

(Sumber: <https://code.visualstudio.com>)

2.2.3 Postman

Postman adalah platform kolaboratif untuk membangun, menguji, dan mengelola API secara *end-to-end*. Postman menyediakan fitur lengkap mulai dari desain API dengan Spec Hub, pembuatan *mock server*, validasi perilaku API, hingga dokumentasi yang dapat di-*host* dan dibagikan. Pengembang dapat menyusun *Collections* untuk mengorganisasi permintaan API, menggunakan *Workspaces* untuk kolaborasi tim, serta memanfaatkan *Flows* untuk membuat alur kerja visual. Postman juga mendukung eksekusi otomatis dengan *Collection Runner* dan Postman CLI, yang memungkinkan pengujian API dijalankan langsung dari *command line*.



POSTMAN

Gambar 2. 3 Postman

Selain fungsi inti, Postman memiliki kemampuan observasi dan manajemen API seperti *Monitors* untuk menguji performa *endpoint*, *Insights* untuk analisis, serta dukungan distribusi API baik internal, partner, maupun publik. Integrasi AI melalui *Postbot* dan *AI Agent Builder* membantu mempercepat debugging, penulisan tes, dan otomatisasi tugas API. Dengan dukungan *Model Context Protocol* (MCP), Postman dapat dihubungkan dengan agen AI maupun *tool* eksternal untuk memperluas fungsionalitas. Postman diadopsi oleh lebih dari 40 juta pengguna dan 500 ribu perusahaan, menjadikannya standar *de facto* dalam pengembangan API modern untuk meningkatkan kolaborasi, mempercepat *time-to-first-API call*, dan menjaga kualitas API.

(Sumber: <https://www.postman.com/>)

2.3 Dasar Teori

2.3.1 *Application Programming Interface (API)*

API (*Application Programming Interface*) adalah mekanisme yang memungkinkan dua perangkat lunak berbeda untuk saling berkomunikasi menggunakan protokol dan aturan tertentu. Melalui API, aplikasi dapat bertukar data dan menjalankan fungsi tanpa harus mengetahui detail internal masing-masing sistem. Contohnya, aplikasi cuaca di ponsel mengambil data dari server BMKG melalui API, lalu menampilkan informasi cuaca kepada pengguna. API memiliki berbagai jenis seperti SOAP, RPC, WebSocket, hingga REST yang menjadi standar paling populer karena fleksibilitasnya, bersifat stateless, serta menggunakan HTTP untuk komunikasi.



Selain mempermudah integrasi antar sistem, API juga membawa banyak keuntungan seperti mempercepat pengembangan, mendorong inovasi, memperluas jangkauan layanan, dan memudahkan pemeliharaan aplikasi. API dapat bersifat privat, publik, partner, atau komposit tergantung kebutuhan penggunaannya. Keamanan API dijaga melalui autentikasi token dan kunci API, sementara endpoint menjadi titik penting dalam komunikasi dan perlu dipantau agar aman serta optimal. Saat ini, API juga terus berkembang dengan adanya GraphQL yang lebih fleksibel dalam permintaan data, serta layanan manajemen API seperti Amazon API Gateway dan AWS AppSync yang membantu perusahaan membangun, mengelola, dan mengamankan API dalam skala besar.

(Sumber: <https://aws.amazon.com/id/what-is/api/>)

2.3.2 **Express**

Express adalah *framework* web aplikasi berbasis Node.js yang *open-source*, ringan, dan minimalis, dirilis pertama kali pada 2010 oleh TJ Holowaychuk. *Framework* ini dirancang untuk mempercepat pengembangan aplikasi web maupun API dengan struktur sederhana namun tetap *powerful*. Express mendukung sistem *routing* yang memungkinkan pengaturan berbagai *endpoint* sesuai kebutuhan, serta sepenuhnya kompatibel dengan ekosistem NPM, sehingga pengembang dapat menambahkan modul eksternal dengan mudah. Sifat modular dan

fleksibel dari Express membuatnya banyak digunakan dalam membangun RESTful API maupun arsitektur *microservices*.



Gambar 2. 4 Express

Keunggulan teknis Express ada pada *middleware*, yaitu fungsi berlapis yang memproses *request* dan *response*, misalnya untuk otentikasi, *logging*, validasi data, atau *error handling*. *Middleware* juga mendukung integrasi dengan *database*, *template engine*, hingga layanan eksternal. Express menyediakan mekanisme penanganan *error* yang terpusat, serta memiliki komunitas besar dengan dokumentasi dan modul siap pakai. Implementasi teknis umumnya mencakup instalasi dengan `npm install express`, pembuatan server menggunakan `app.listen()`, *routing* dengan `app.get()` atau `app.post()`, dan *middleware* kustom melalui `app.use()`. Dengan kombinasi kesederhanaan sintaks, fleksibilitas *middleware*, serta ekosistem luas, Express menjadi salah satu *framework backend* paling populer untuk aplikasi web modern.

(Sumber: <https://www.codepolitan.com/blog/apa-itu-express-js-pengertian-manfaat-contoh-syntax/>)

2.3.3 JavaScript

JavaScript adalah bahasa pemrograman tingkat tinggi yang awalnya dikembangkan untuk membuat halaman web lebih interaktif. Berjalan di sisi klien (*client-side*) melalui *browser*, JavaScript kini juga dapat berjalan di sisi server berkat *platform* seperti Node.js. Bahasa ini bersifat *interpreted*, *dynamic*, dan *event-driven*, dengan dukungan *object-oriented* dan *functional programming*. JavaScript memiliki *runtime non-blocking* berbasis *event loop* yang memungkinkan eksekusi asinkron, sehingga sangat cocok untuk aplikasi *real-time*. Standarisasi JavaScript dikelola oleh ECMAScript (ES), dengan versi terbaru membawa fitur modern seperti *let/const*, *arrow function*, *async/await*, dan modularisasi.



Gambar 2. 5 JavaScript

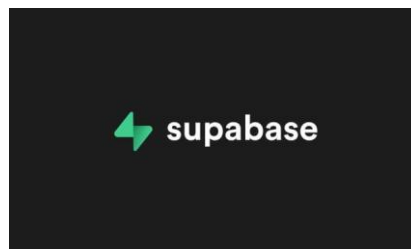
Secara teknis, JavaScript mendukung DOM Manipulation untuk mengubah struktur HTML/CSS secara langsung, *event handling* untuk merespons aksi pengguna, dan AJAX/fetch

API untuk komunikasi asinkron dengan server. Dalam konteks *backend*, JavaScript melalui Node.js memungkinkan penggunaan modul bawaan seperti `http` untuk membuat server, serta dukungan ekosistem NPM dengan jutaan paket. JavaScript juga menjadi fondasi *framework* populer seperti React, Vue, Angular, dan Express. Kombinasi kemampuan *front-end* dan *back-end* menjadikan JavaScript sebagai bahasa pemrograman *full-stack*, memungkinkan *developer* membangun aplikasi web modern dari sisi antarmuka hingga logika server dengan satu bahasa yang konsisten.

(Sumber: <https://web.dev/javascript?hl=id>)

2.3.4 Supabase

Supabase adalah platform *open-source* berbasis PostgreSQL yang menyediakan layanan *backend* siap pakai untuk aplikasi modern. Secara teknis, Supabase menawarkan *Database* dengan dukungan *real-time*, *backup* otomatis, serta ekstensi Postgres; *Authentication* untuk *login* berbasis *email/password*, *OAuth*, *passwordless*, hingga *mobile*; *Storage* untuk menyimpan dan mengelola file besar dengan kebijakan keamanan berbasis *Row Level Security* (RLS); serta *Realtime API* untuk mendengarkan perubahan data dan sinkronisasi *state* antar klien. Selain itu, tersedia *Edge Functions* yang memungkinkan eksekusi fungsi *server-side* secara global dengan latensi rendah, serta fitur tambahan seperti *cron*, *queues*, dan *AI vector database*.



Gambar 2. 6 Supabase

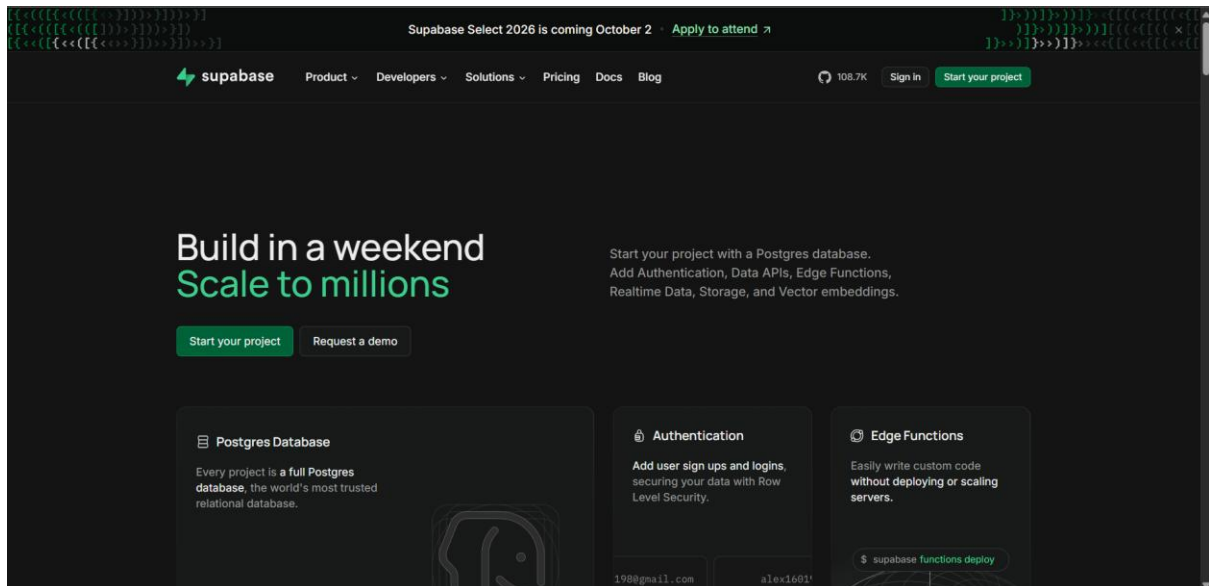
Dari sisi integrasi teknis, Supabase menyediakan *client libraries* untuk berbagai bahasa pemrograman seperti JavaScript, Flutter, Python, C#, Swift, dan Kotlin. *Developer* dapat menggunakan Supabase CLI untuk manajemen proyek, *deployment*, hingga *self-hosting*. Supabase juga kompatibel dengan layanan populer seperti Firebase (Auth, Storage, Firestore), MySQL, MSSQL, Neon, Heroku, dan Vercel Postgres, sehingga memudahkan migrasi sistem yang sudah ada. Dengan kombinasi *database*, *auth*, *storage*, *realtime*, dan *edge functions*, Supabase berfungsi sebagai alternatif *Backend-as-a-Service* (BaaS) yang memungkinkan pengembangan aplikasi cepat tanpa harus membangun infrastruktur *backend* dari nol.

(Sumber : <https://supabase.com/docs>)

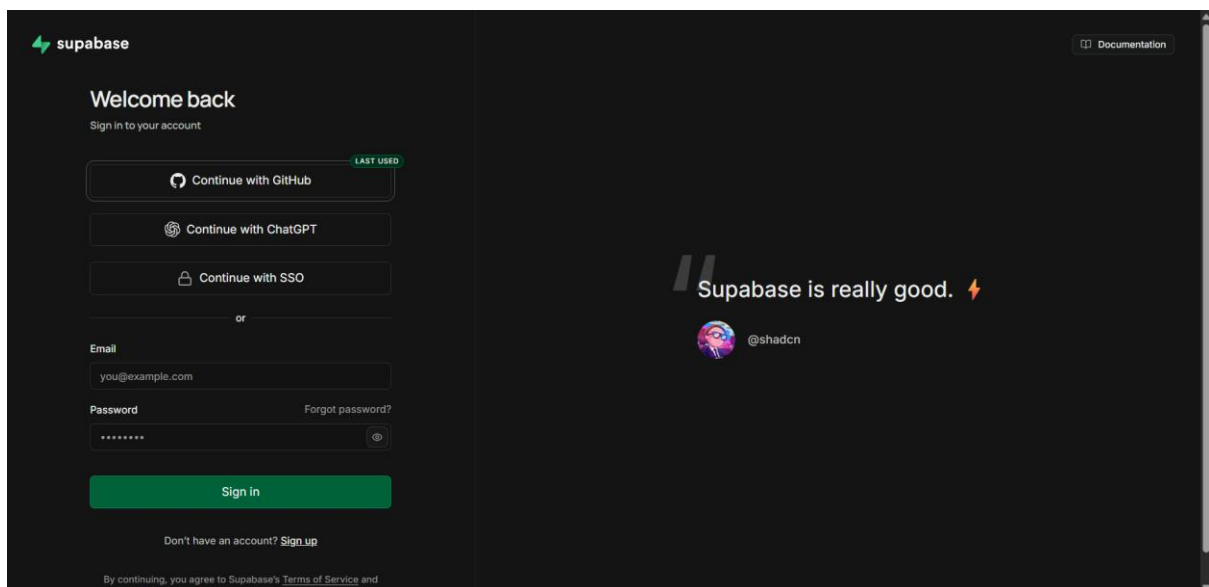
2.4 Langkah Percobaan

2.4.1 Percobaan 1 (Pembuatan API)

1. Kunjungi situs web Supabase (<https://supabase.com/>).



2. Daftar atau masuk ke akun Anda.



3. Klik "New project" atau "New organization" dan ikuti instruksi untuk membuat proyek baru. Berikan nama proyek yang relevan, seperti Sales-API.

Create a new organization
Organizations are a way to group your projects. Each organization can be configured with different team members and billing settings.

Name:
What's the name of your company or team? You can change this later.

Type:
What best describes your organization?

Plan:
Which plan fits your organization's needs best? [Learn more.](#)

[Cancel](#) [Create organization](#)

Supabase Select 2026
A curated day of talks by the industry's best builders. Join us on October 2nd in San Francisco.
[Apply to attend](#)

GitHub (optional) [Connect GitHub](#)
Ideal for agent-first workflows. Update your schema in code and push it to GitHub. Supabase deploys the changes. [Learn more](#)

Project name:

Database password: [Copy](#)
This password is strong. [Generate a password.](#)

Region:
Select the region closest to your users for the best performance.

Security

- ☒ **Enable Data API**
Autogenerate a RESTful API for your public schema. Recommended if using a client library like [supabase-js](#).
- ☒ **Automatically expose new tables**
Grants privileges to Data API roles by default, exposing new tables. We recommend disabling this to control access manually.
- ☐ **Enable automatic RLS**
Create an event trigger that automatically enables Row Level Security on all new tables in the public schema.

[ADVANCED CONFIGURATION >](#)

Supabase Select 2026
A curated day of talks by the industry's best builders. Join us on October 2nd in San Francisco.
[Apply to attend](#)

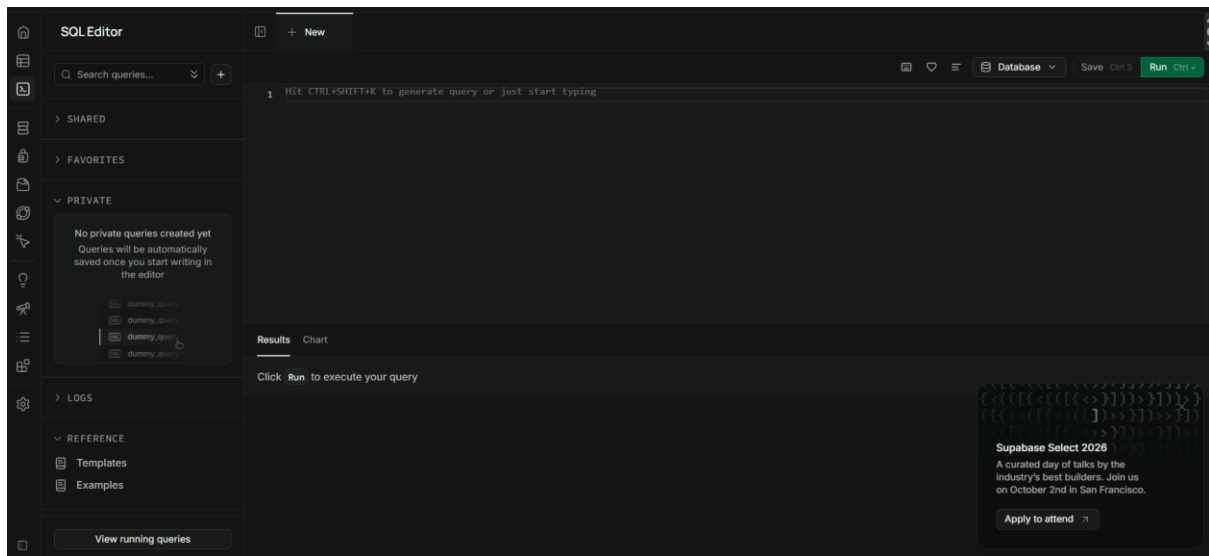
- Setelah berhasil membuat proyek baru di Supabase, buka SQL Editor melalui ikon di sidebar, lalu klik "+ New query" untuk mulai membuat tabel menggunakan SQL.

Project Overview
Table Editor
SQL Editor [Go to SQL Editor](#) [then S](#)

Database: [Copy](#)

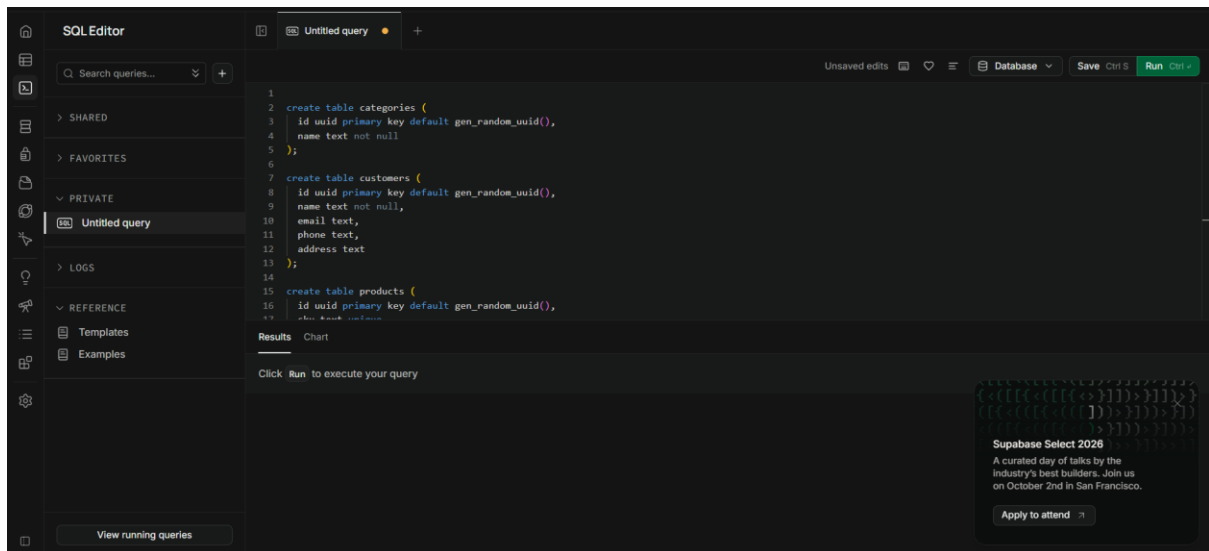
Primary Database: Northeast Asia (Seoul) ap-northeast-2 · 13a.nano
CPU 0% · Disk 13% · RAM 46% · 4/80 conns

Supabase Select 2026
A curated day of talks by the industry's best builders. Join us on October 2nd in San Francisco.
[Apply to attend](#)

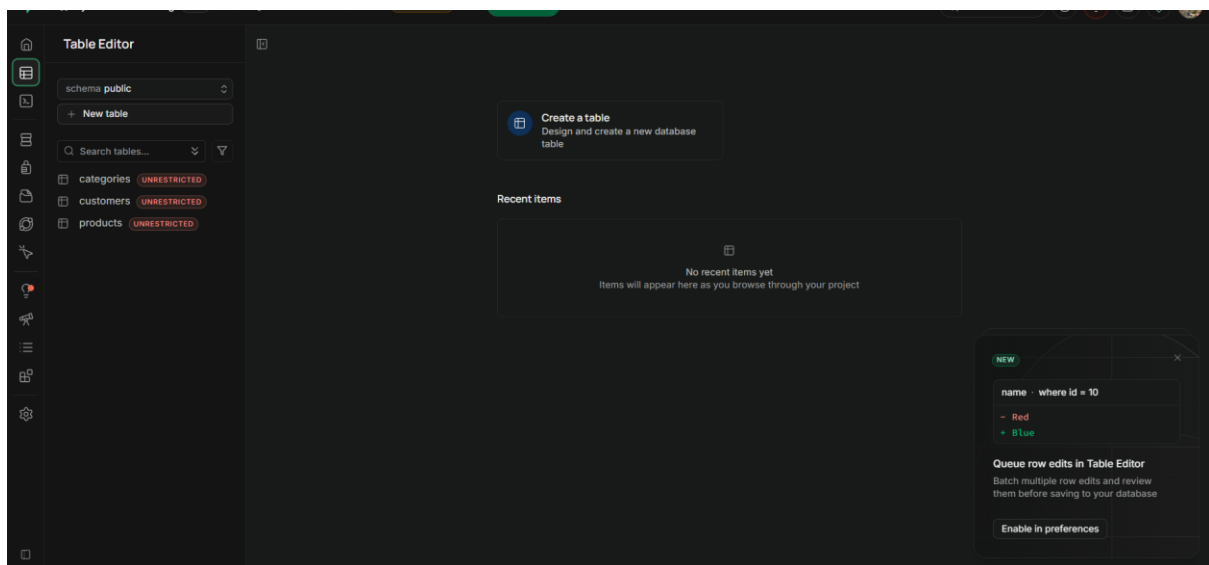


5. Salin skema SQL berikut ke dalam *editor*, jalankan seluruh blok kode sekaligus karena tabel *Product* bergantung pada tabel *Categories*, lalu klik "RUN" hingga proses berhasil.

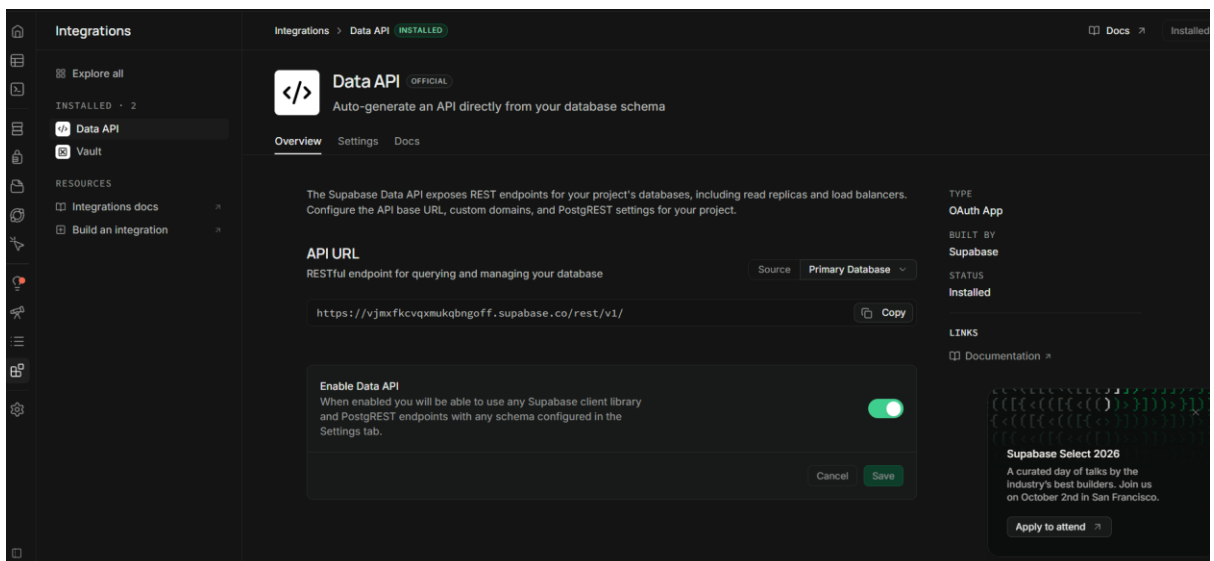
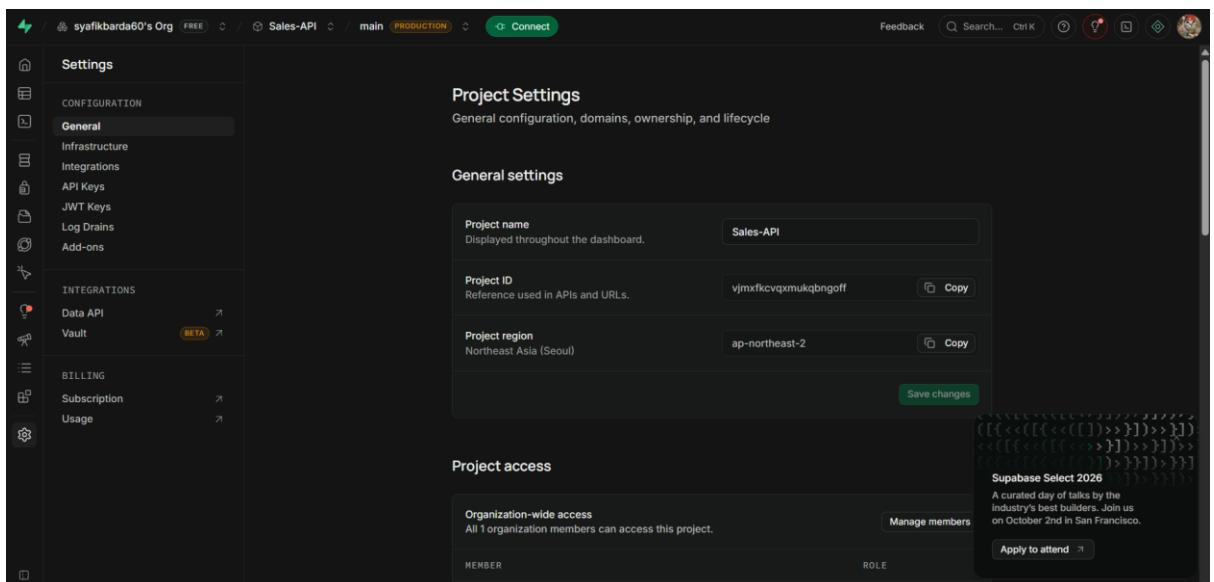
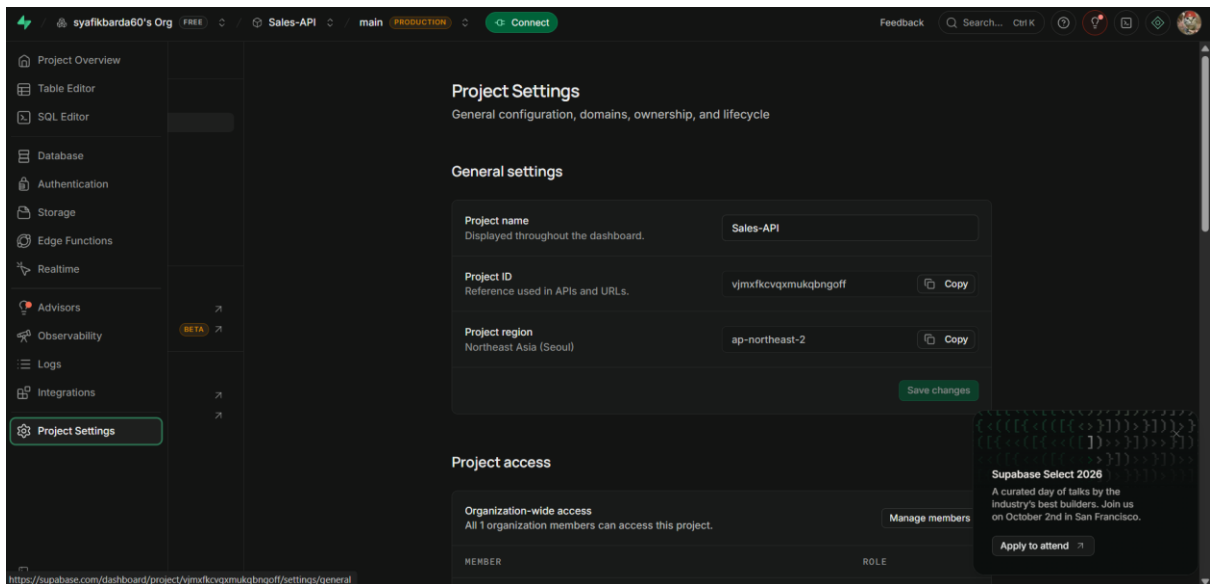
```
create table categories (  
  id uuid primary key default gen_random_uuid(),  
  name text not null  
);  
  
create table customers (  
  id uuid primary key default gen_random_uuid(),  
  name text not null,  
  email text,  
  phone text,  
  address text  
);  
  
create table products (  
  id uuid primary key default gen_random_uuid(),  
  sku text unique,  
  name text not null,  
  description text,  
  category_id uuid references categories(id),  
  price numeric(12,2) default 0,  
  stock integer default 0  
);
```

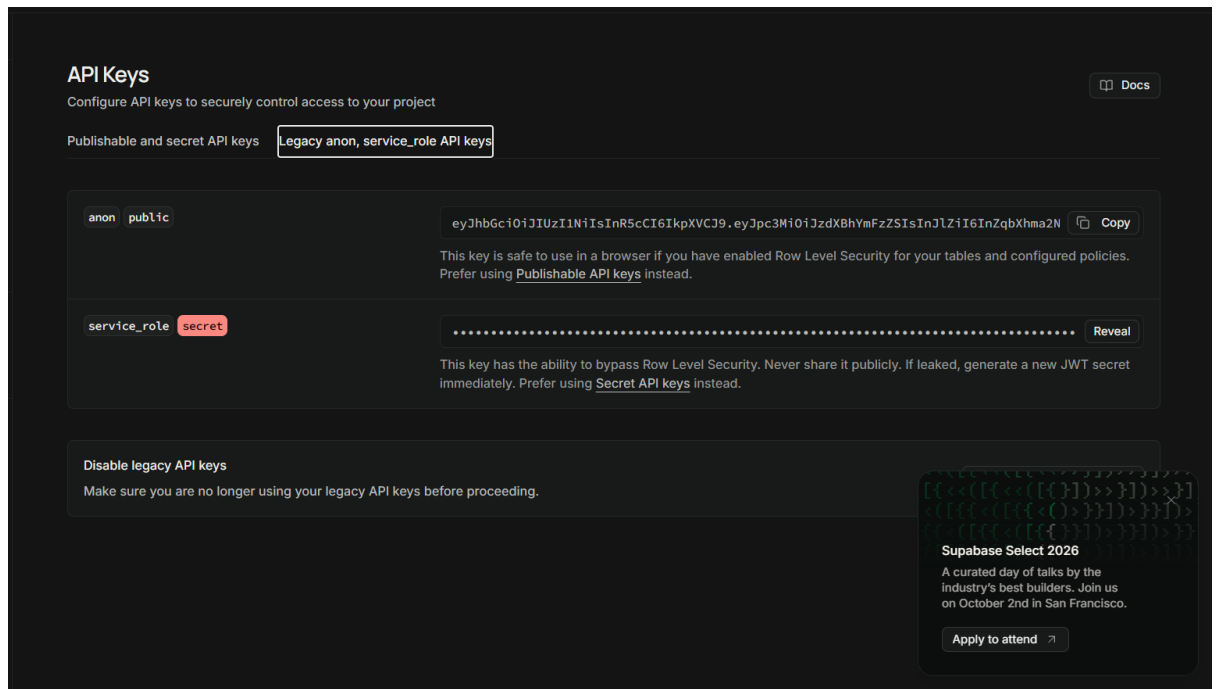


6. Verifikasi tabel dengan membuka *Table Editor* di *sidebar* setelah *query* berhasil dijalankan, lalu pastikan tiga tabel baru (*categories*, *Customer*, dan *Products*) telah terbentuk; periksa struktur kolom pada masing-masing tabel, pastikan relasi *Foreign Keys* dengan tabel *categories* dan *Products* sudah terlihat.



7. Dapatkan kredensial API dengan masuk ke *Project Settings*, lalu pilih *Data API* untuk menyalin *Project URL*, dan buka *API Keys* untuk menyalin *anon public key* sebagai nilai *SUPABASE_URL* dan *SUPABASE_KEY* yang akan digunakan pada konfigurasi selanjutnya.





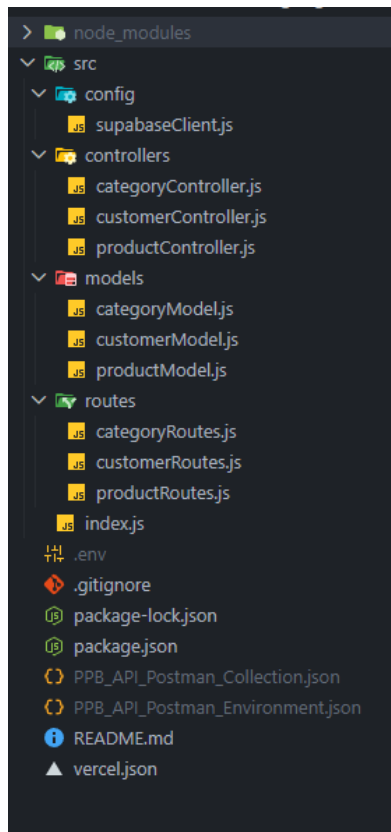
8. Buat folder proyek baru di direktori lokal Anda. Beri nama apa saja, misalnya *Sales-API*.
9. Buka terminal atau *command prompt* di dalam folder tersebut.
10. Jalankan perintah berikut untuk menginisialisasi proyek Node.js dan membuat file *package.json*.

```
npm init -y
```

11. Sekarang, pasang semua pustaka yang dibutuhkan untuk proyek. Pustaka ini diperlukan untuk menjalankan server, terhubung ke Supabase, dan mengelola variabel lingkungan.

```
npm install @supabase/supabase-js dotenv express  
npm install --save-dev nodemon
```

12. Masuk ke Visual Studio Code dengan mengetikkan code . di terminal atau *command prompt*.
13. Buat struktur folder dan file seperti yang terlihat pada gambar berikut.



14. Tambahkan kode ini ke `src/config/supabaseClient.js`. File ini akan menginisialisasi klien Supabase menggunakan kredensial dari file `.env`.

```
import dotenv from "dotenv";
import { createClient } from "@supabase/supabase-js";

dotenv.config();

const supabaseUrl = process.env.SUPABASE_URL;
const supabaseKey = process.env.SUPABASE_KEY;

export const supabase = createClient(supabaseUrl, supabaseKey);
```

15. Tambahkan kode-kode ini ke dalam folder `src/models`. Masing-masing file model akan berisi logika untuk berinteraksi dengan tabel yang sesuai di *database* Supabase.

`src/models/categoryModel.js`

```
import { supabase } from "../config/supabaseClient.js";

export const CategoryModel = {
  async create(name) {
    const { data, error } = await supabase
      .from("categories")
      .insert([ { name } ])
      .select()
      .single();
    if (error) throw error;
    return data;
  },
}
```

```

    async getAll() {
      const { data, error } = await
supabase.from("categories").select("*");
      if (error) throw error;
      return data;
    },

    async getById(id) {
      const { data, error } = await supabase
        .from("categories")
        .select("*")
        .eq("id", id)
        .single();
      if (error) throw error;
      return data;
    },

    async update(id, name) {
      const { data, error } = await supabase
        .from("categories")
        .update({ name })
        .eq("id", id)
        .select()
        .single();
      if (error) throw error;
      return data;
    },

    async remove(id) {
      const { error } = await supabase.from("categories").delete().eq("id",
id);
      if (error) throw error;
      return { message: "Category deleted" };
    },
  };

```

src/models/customerModel.js

```

import { supabase } from "../config/supabaseClient.js";

export const CustomerModel = {
  async getAll() {
    const { data, error } = await supabase.from("customers").select("*");
    if (error) throw error;
    return data;
  },

  async getById(id) {
    const { data, error } = await supabase
      .from("customers")
      .select("*")
      .eq("id", id)
      .single();
    if (error) throw error;
    return data;
  },
};

```

```

    async create(customer) {
      const { data, error } = await supabase
        .from("customers")
        .insert([customer])
        .select()
        .single();
      if (error) throw error;
      return data;
    },

    async update(id, customer) {
      const { data, error } = await supabase
        .from("customers")
        .update(customer)
        .eq("id", id)
        .select()
        .single();
      if (error) throw error;
      return data;
    },

    async remove(id) {
      const { error } = await supabase.from("customers").delete().eq("id",
id);
      if (error) throw error;
      return { message: "Customer deleted successfully" };
    },
  };
};

```

src/models/productModel.js

```

import { supabase } from "../config/supabaseClient.js";

export const ProductModel = {
  async getAll() {
    const { data, error } = await supabase
      .from("products")
      .select(
        "id, sku, name, description, price, stock, category_id"
      );
    if (error) throw error;
    return data;
  },

  async getById(id) {
    const { data, error } = await supabase
      .from("products")
      .select(
        `
        id, sku, name, description, price, stock,
        categories ( id, name )
      `
      )
      .eq("id", id)
      .single();
    if (error) throw error;
    return data;
  }
};

```



```

    },

    async create(payload) {
      const { data, error } = await supabase
        .from("products")
        .insert([payload])
        .select();
      if (error) throw error;
      return data[0];
    },

    async update(id, payload) {
      const { data, error } = await supabase
        .from("products")
        .update(payload)
        .eq("id", id)
        .select();
      if (error) throw error;
      return data[0];
    },

    async remove(id) {
      const { error } = await supabase.from("products").delete().eq("id",
id);
      if (error) throw error;
      return { message: "Product deleted successfully" };
    },
  };

```

16. Tambahkan kode-kode ini ke dalam *folder* `src/controllers`. *Controller* akan memproses permintaan HTTP dan menggunakan model untuk berinteraksi dengan *database*.

`src/controllers/categoryController.js`

```

import { CategoryModel } from "../models/categoryModel.js";

export const CategoryController = {
  async create(req, res) {
    try {
      const { name } = req.body;
      const category = await CategoryModel.create(name);
      res.status(201).json(category);
    } catch (err) {
      res.status(400).json({ error: err.message });
    }
  },

  async getAll(req, res) {
    try {
      const categories = await CategoryModel.getAll();
      res.json(categories);
    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  },

  async getById(req, res) {

```

```

    try {
      const { id } = req.params;
      const category = await CategoryModel.getById(id);
      res.json(category);
    } catch (err) {
      res.status(404).json({ error: err.message });
    }
  },

  async update(req, res) {
    try {
      const { id } = req.params;
      const { name } = req.body;
      const category = await CategoryModel.update(id, name);
      res.json(category);
    } catch (err) {
      res.status(400).json({ error: err.message });
    }
  },

  async remove(req, res) {
    try {
      const { id } = req.params;
      const result = await CategoryModel.remove(id);
      res.json(result);
    } catch (err) {
      res.status(400).json({ error: err.message });
    }
  },
};

```

src/controllers/customerController.js

```

import { CustomerModel } from "../models/customerModel.js";

export const CustomerController = {
  async getAll(req, res) {
    try {
      const customers = await CustomerModel.getAll();
      res.json(customers);
    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  },

  async getById(req, res) {
    try {
      const customer = await CustomerModel.getById(req.params.id);
      res.json(customer);
    } catch (err) {
      res.status(404).json({ error: err.message });
    }
  },

  async create(req, res) {
    try {
      const customer = await CustomerModel.create(req.body);
      res.status(201).json(customer);
    } catch (err) {

```

```

        res.status(400).json({ error: err.message });
    },

    async update(req, res) {
        try {
            const customer = await CustomerModel.update(req.params.id,
req.body);
            res.json(customer);
        } catch (err) {
            res.status(400).json({ error: err.message });
        }
    },

    async remove(req, res) {
        try {
            await CustomerModel.remove(req.params.id);
            res.json({ message: "Customer deleted successfully" });
        } catch (err) {
            res.status(400).json({ error: err.message });
        }
    },
};

```

src/controllers/productController.js

```

import { ProductModel } from "../models/productModel.js";

export const ProductController = {
    async getAll(req, res) {
        try {
            const products = await ProductModel.getAll();
            res.json(products);
        } catch (err) {
            res.status(500).json({ error: err.message });
        }
    },

    async getById(req, res) {
        try {
            const product = await ProductModel.getById(req.params.id);
            res.json(product);
        } catch (err) {
            res.status(404).json({ error: err.message });
        }
    },

    async create(req, res) {
        try {
            const product = await ProductModel.create(req.body);
            res.status(201).json(product);
        } catch (err) {
            res.status(400).json({ error: err.message });
        }
    },

    async update(req, res) {
        try {
            const product = await ProductModel.update(req.params.id, req.body);
            res.json(product);
        }
    },
};

```

```

    } catch (err) {
      res.status(400).json({ error: err.message });
    }
  },

  async remove(req, res) {
    try {
      await ProductModel.remove(req.params.id);
      res.json({ message: "Product deleted successfully" });
    } catch (err) {
      res.status(400).json({ error: err.message });
    }
  },
};

```

17. Tambahkan kode-kode ini ke dalam folder `src/routes`. *Router* akan memetakan URL ke fungsi *controller* yang sesuai.

src/routes/categoryRoutes.js

```

import express from "express";
import { CategoryController } from
"../controllers/categoryController.js";

const router = express.Router();

router.post("/", CategoryController.create);
router.get("/", CategoryController.getAll);
router.get("/:id", CategoryController.getById);
router.put("/:id", CategoryController.update);
router.delete("/:id", CategoryController.remove);

export default router;

```

src/routes/customerRoutes.js

```

import express from "express";
import { CustomerController } from
"../controllers/customerController.js";

const router = express.Router();

router.get("/", CustomerController.getAll);
router.get("/:id", CustomerController.getById);
router.post("/", CustomerController.create);
router.put("/:id", CustomerController.update);
router.delete("/:id", CustomerController.remove);

export default router;

```

src/routes/productRoutes.js

```

import express from "express";
import { ProductController } from "../controllers/productController.js";

const router = express.Router();

router.get("/", ProductController.getAll);

```

```
router.get("/:id", ProductController.getById);
router.post("/", ProductController.create);
router.put("/:id", ProductController.update);
router.delete("/:id", ProductController.remove);

export default router;
```

18. Tambahkan kode berikut ke `src/index.js`. Ini adalah file utama yang akan menjalankan server.

```
import express from "express";
import dotenv from "dotenv";
import categoryRoutes from "../routes/categoryRoutes.js";
import productRoutes from "../routes/productRoutes.js";
import customerRoutes from "../routes/customerRoutes.js";

dotenv.config();

const app = express();
app.use(express.json());

app.use("/api/categories", categoryRoutes);
app.use("/api/products", productRoutes);
app.use("/api/customers", customerRoutes);

const port = process.env.PORT || 3000;
app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

19. Buat file-file ini di direktori *root* proyek. File `.env` adalah template untuk kredensial Anda, dan `.gitignore` akan memastikan file sensitif tidak diunggah ke repositori.

.env

```
SUPABASE_URL=
SUPABASE_KEY=
PORT=3000
```

.gitignore

```
.env
/node_modules
```

20. Masukkan Project URL dan *anon public key* yang telah diperoleh sebelumnya ke dalam file `.env` dengan menuliskan nilainya masing-masing pada variabel `SUPABASE_URL` dan `SUPABASE_KEY`.
21. Pastikan file `package.json` Anda memiliki konfigurasi seperti berikut untuk menjalankan proyek.

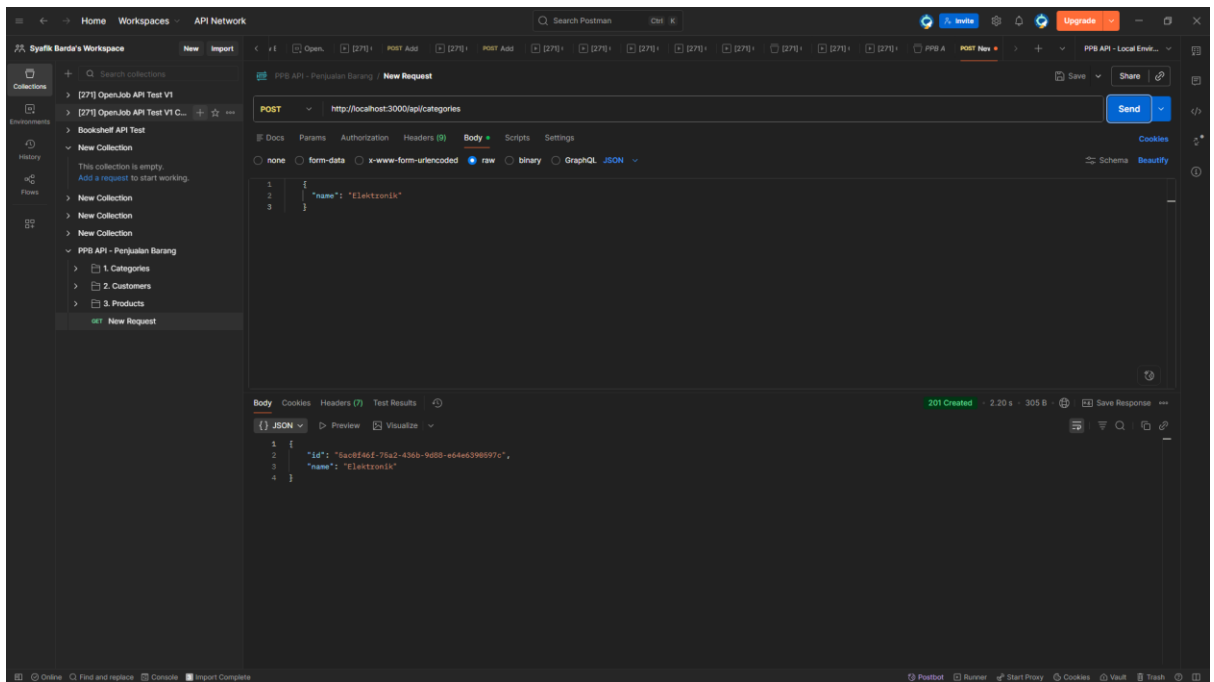
```
{
  "name": "ppb_api",
```

```
"version": "1.0.0",
"main": "index.js",
"type": "module",
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "start": "node src/index.js",
  "dev": "nodemon src/index.js"
},
"keywords": [],
"author": "",
"license": "ISC",
"description": "",
"dependencies": {
  "@supabase/supabase-js": "^2.113.0",
  "dotenv": "^17.4.2",
  "express": "^5.2.1"
},
"devDependencies": {
  "nodemon": "^3.1.14"
}
}
```

22. Tambahkan *source code* berikut ke dalam `vercel.json`

```
{
  "version": 2,
  "builds": [
    {
      "src": "src/index.js",
      "use": "@vercel/node"
    }
  ],
  "routes": [
    {
      "src": "/*.*",
      "dest": "src/index.js"
    }
  ]
}
```

23. Buka terminal di folder proyek dan jalankan perintah `npm run dev`. Jika berhasil, Anda akan melihat pesan seperti *"Server running on port 3000"*.
24. Saatnya pengujian POST pada routes `/categories`



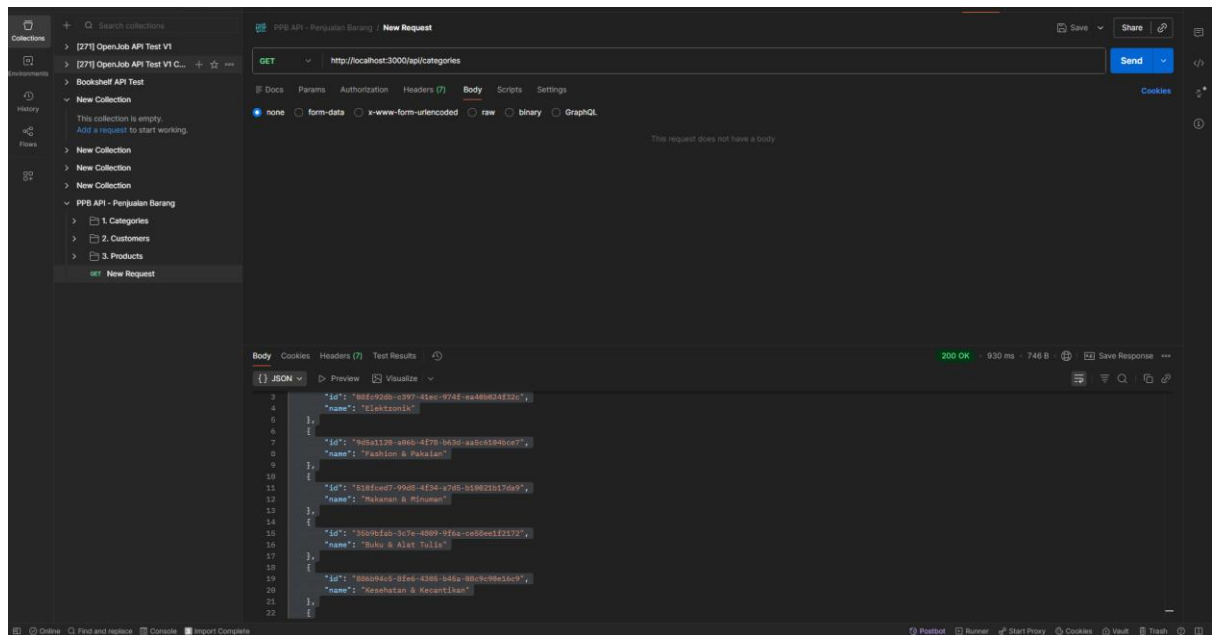
Disini kita perlu memasukkan routes-nya lalu masukkan *body raw*-nya dengan

```
{
  "name": "Elektronik"
}
```

Makan akan ada *body respond* dibawahnya jika berhasil.

```
{
  "id": "5ac0f46f-75a2-436b-9d88-e64e6390597c",
  "name": "Elektronik"
}
```

25. Selanjutnya pengujian GET pada *routes /categories*



Maka *respond JSON*-nya akan menampilkan *categories* yang sudah di buat pada *endpoint* sebelumnya adalah:

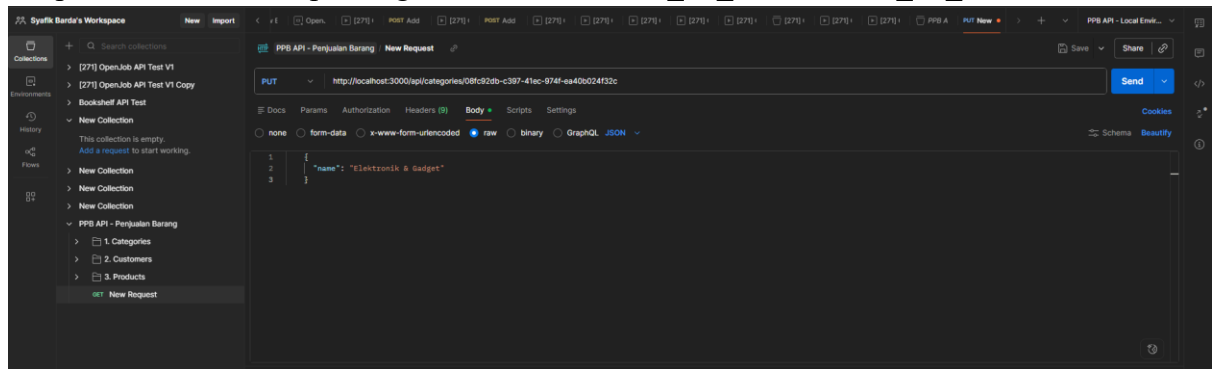
```

[
  {
    "id": "08fc92db-c397-41ec-974f-ea40b024f32c",
    "name": "Elektronik"
  },
  {
    "id": "9d5a1120-a06b-4f78-b63d-aa5c6104bce7",
    "name": "Fashion & Pakaian"
  },
  {
    "id": "510fced7-99d8-4f34-a7d5-b10021b17da9",
    "name": "Makanan & Minuman"
  },
  {
    "id": "35b9bfab-3c7e-4809-9f6a-ce58ee1f2172",
    "name": "Buku & Alat Tulis"
  },
  {
    "id": "886b94c5-8fe6-4305-b45a-08c9c90e16c9",
    "name": "Kesehatan & Kecantikan"
  },
  {
    "id": "e6984d46-4141-408a-bcf7-3f23567582c7",
    "name": "Perlengkapan Rumah"
  },
  {
    "id": "1f45d14b-9e5a-4aff-b789-5e9663522dce",
    "name": "Olahraga & Hobi"
  }
]

```

26. Selanjutnya, pengujian method PUT pada *routes* `/categories`, untuk melakukan pengujian ini, jangan lupa untuk menambahkan ID *catégorie* yang ingin diubah, berikut adalah contoh *endpoint url*-nya :

“`http://localhost:3000/api/categories/MASUKKAN_ID_KATEGORI_DI_SINI`”



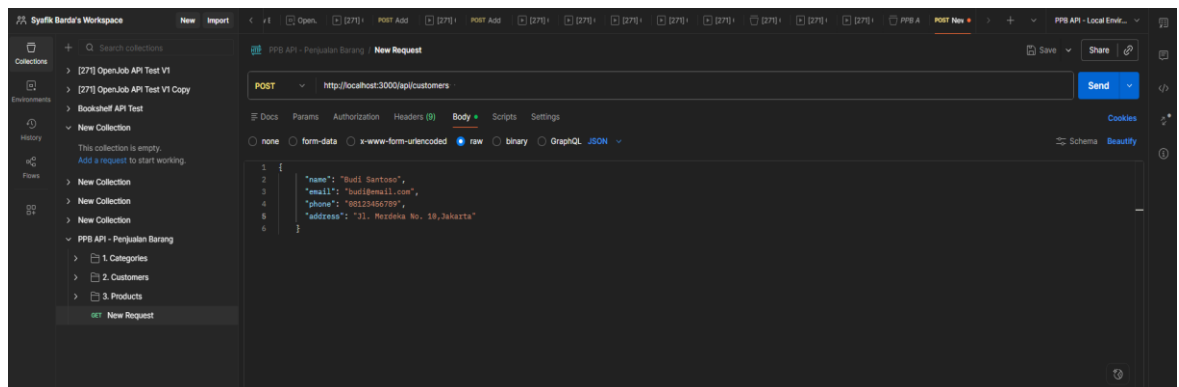
Dan masukan *body raw*-nya dengan format sebagai berikut :

```
{
  "name": "Elektronik & Gadget"
}
```

Maka *respond* JSON-nya akan menampilkan *categories* yang sudah diubah :

```
{
  "id": "08fc92db-c397-41ec-974f-ea40b024f32c",
  "name": "Elektronik & Gadget"
}
```

27. Selanjutnya kita coba POST *routes* /customers untuk mencoba menambahkan data *customer*



Masukan *body raw*-nya dengan format berikut

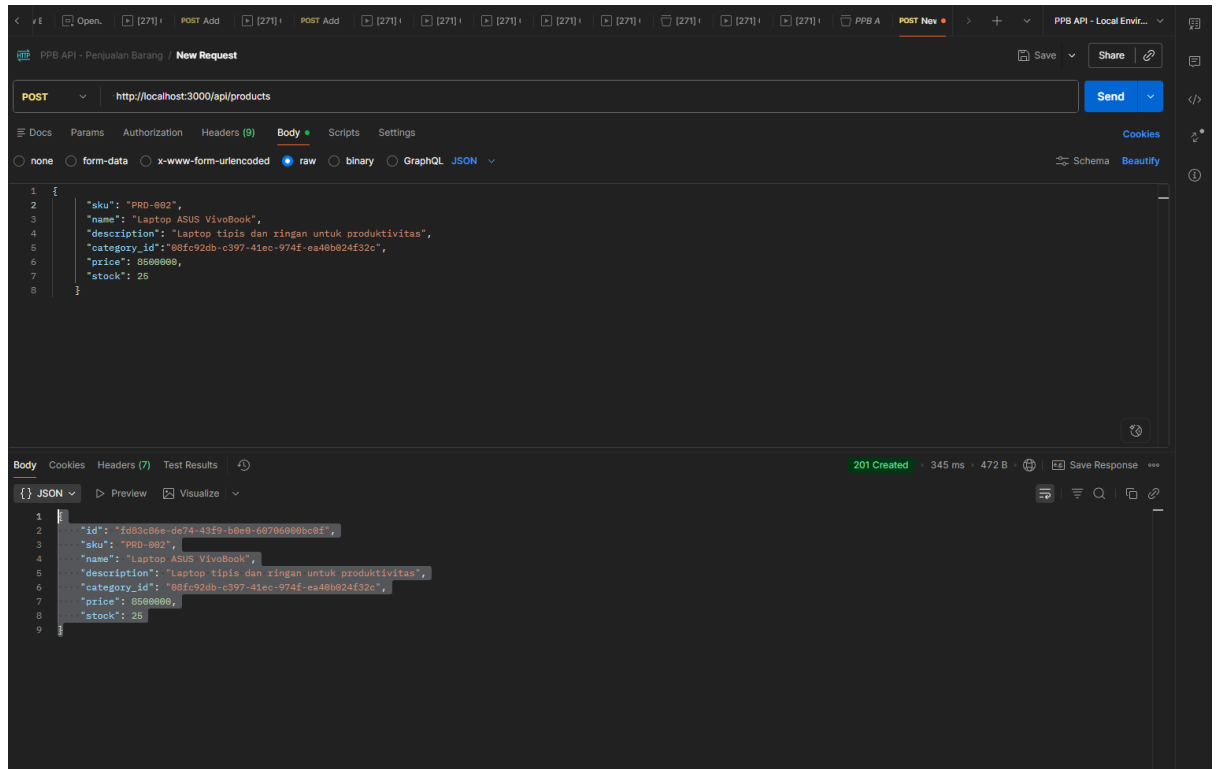
```
{
  "name": "Budi Santoso",
  "email": "budi@email.com",
  "phone": "08123456789",
  "address": "Jl. Merdeka No. 10, Jakarta"
}
```

Maka hasil outputnya adalah sebagai berikut :

```
{
  "id": "a99b9ccf-18ba-4d80-a26a-71397bc608e1",
  "name": "Budi Santoso",
  "email": "budi@email.com",
  "phone": "08123456789",
  "address": "Jl. Merdeka No. 10, Jakarta"
}
```

```
}
```

28. Selanjutnya adalah pengujian dengan menggunakan method POST untuk *routes* /products, untuk melakukan pengujian ini, pastikan memasukan id yang valid pada JSON-nya



body raw-nya untuk pengujian ini adalah sebagai berikut

```
{  "sku": "PRD-002",  "name": "Laptop ASUS VivoBook",  "description": "Laptop tipis dan ringan untuk produktivitas",  "category_id": "08fc92db-c397-41ec-974f-ea40b024f32c",  "price": 8500000,  "stock": 25}
```

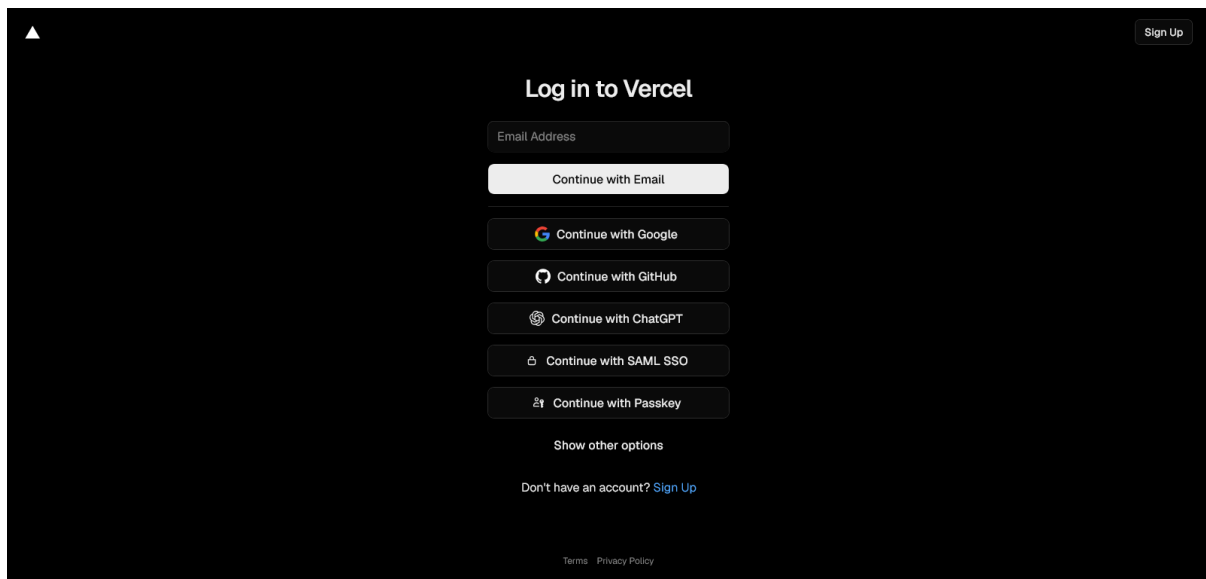
Hasilnya:

```
{  "id": "fd83c86e-de74-43f9-b0e0-60706000bc0f",  "sku": "PRD-002",  "name": "Laptop ASUS VivoBook",  "description": "Laptop tipis dan ringan untuk produktivitas",  "category_id": "08fc92db-c397-41ec-974f-ea40b024f32c",  "price": 8500000,  "stock": 25}
```

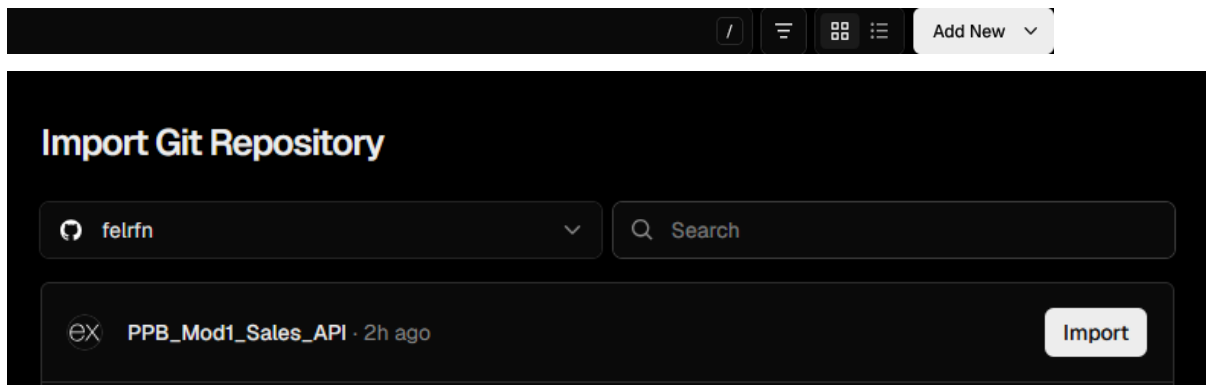
29. Misal ingin melihat / *import* bisa lihat *repository* berikut https://github.com/syafikbarda60/PPB_Mod1_Sales_API.git dengan membaca README.md. Jadi hanya perlu *import collections* pada postman.

2.4.2 Percobaan 2 (Deployment API)

1. Unggah Proyek ke Repositori Git.
2. *Login* ke Vercel: Buka <https://vercel.com/>.



3. Buka *dashboard* lalu klik "*Add New...*", pilih "*Project*", cari repositori tempat kode API disimpan, dan klik "*Import*".



4. Setelah repositori berhasil diimpor, Vercel akan mendeteksi proyek sebagai Node.js dan otomatis mengatur *Framework Preset*, serta pastikan *Root Directory* sesuai dengan lokasi kode Anda.

New Project

Importing from GitHub
 felrnf/PPB_Mod1_Sales_API ↗ main

Choose where you want to create the project and give it a name.

Vercel Team: fel's projects (Hobby) Project Name: ppb-mod1-sales-api

Application Preset: EX Express

Root Directory: ./ Edit

- Selanjutnya, pada bagian *Environment Variables*, tambahkan kredensial Supabase dengan membuat variabel `SUPABASE_URL` berisi *Project URL* dan variabel `SUPABASE_KEY` berisi *anon key*, lalu klik *Add* untuk menyimpan.

Environment Variables

Key: SUPABASE_URL

Value: [masked]

Environments: Production and Preview

Key: SUPABASE_KEY

Value: [masked]

Environments: Production and Preview

Import .env or paste the .env contents [Learn more](#) + Add More

Deploy

- klik *"Deploy"* setelah menambahkan variabel *environment*, lalu tunggu proses *build* berjalan hingga selesai dengan *log* yang dapat dipantau secara *real-time*.

7. Kemudian, setelah *deployment* berhasil, Vercel akan menampilkan URL publik yang bisa diakses melalui tombol "*Visit*".
8. Terakhir, lakukan verifikasi API dengan menguji *endpoint* melalui browser atau Postman menggunakan URL publik yang diberikan untuk memastikan API berjalan dengan benar dan menampilkan data JSON dari Supabase.

2.5 Kesimpulan

1. Implementasi API ini membuktikan bahwa prinsip RESTful dapat diterapkan secara efektif untuk operasi data. Penggunaan metode HTTP standar (*GET*, *POST*, *PUT*, *DELETE*) secara langsung merefleksikan operasi CRUD pada sumber daya, menghasilkan antarmuka yang intuitif dan mudah dipahami.
2. Pemisahan kode ke dalam model dan *controller* membuat pengembangan lebih terorganisir dan efisien.
3. Penggunaan Supabase mempercepat proses pengembangan *backend* dengan menyediakan layanan *database* yang mudah diakses.
4. Penerapan arsitektur MVC-like berhasil memisahkan tanggung jawab, membuat kode lebih mudah dikelola.
5. API yang baik harus disusun dengan struktur yang rapi, memanfaatkan variabel lingkungan untuk menjaga keamanan serta fleksibilitas konfigurasi, dan dipersiapkan agar siap di-*deploy* pada berbagai lingkungan produksi.